Your First Validation Function

Module 1: Python Foundations

```
row = "ADD,Uno"
fields = row.split(",")
if len(fields) != 3:
    print("bad format")
```

By the End of Today, You'll Be Able To…

1

Write a validation function from scratch

2

Report exactly which line failed with `enumerate()`

You'll Know These by the End of Today

Batch file

One record per line

Validation

Check data against rules

`enumerate()`

Add each line's position

Reading Recap: What Goes In, What Comes Out

Today's Reading: Your First Validation Function

A school office's sign-up file hides bad lines among good ones. Reopen the reading:
What Goes In, What Comes Out: Designing Your First Validation Function

The Scenario: Class Sign-Up Forms

Each form is one line with a student ID, grade and class, split by `|`.

5 lines, 3 problems: a missing field, grade `"10th"`, grade `14`.

One function checks every line and lists the problems before anyone makes class lists.

READING RECAP

Key Terms and Code From the Reading

VALIDATION

Check each line against the rules before anyone uses it.

STRUCTURE CHECK FIRST

`len(fields) != 3` runs before reading the grade.

VALUE CHECK

`"10th".isdigit()` is False, so `int()` never sees it.

CONTRACT

Takes the form lines and returns a list of problems, empty if all are valid.

READING CHECK

The line `"S-2050||Biology"` has a blank grade. What does `find_form_problems` report?

A "Expected 3 fields"

B "Grade is not a number"

C Nothing. The line is valid.

ANSWER KEY

Answer Key: A Blank Grade Is Not a Number

Correct answer: B. "Grade is not a number"

The line still splits into 3 fields: `['S-2050', '', 'Biology']`. But `"".isdigit()` is False.

READING CHECK

A teammate indents `return problems` inside the `for` loop. What prints first?

A

0 problems found

B

3 problems found

C

An error

ANSWER KEY

Answer Key: Return Inside the Loop

Correct answer: A. 0 problems found

The function returns after line 1, and line 1 is valid. No error, so everyone trusts a bad file.

Your Questions From the Reading

Turn and talk, 1 min:
One question each.

We take 2 or 3 questions now.

Bigger ones wait for today's matching slide.

Now Build the Whole Thing

Loops, conditionals, and functions combine into one real, useful tool.

A Batch File Is Checked One Record at a Time

INPUT

One record per line.

CHECK

Validate each line.

REPORT

Name problems — never crash.

IN THE NEWS

Right Number, Wrong Unit: NASA Lost a Mars Orbiter



Mars Climate Orbiter before a spin test, November 13, 1998. Photo: NASA/JPL, public domain, via Wikimedia Commons

Ground software sent pound-force-seconds
Navigation software expected newton-seconds
The team never checked the file's units

Every value in that file was a valid number.
What else should a check have tested?

Source: NASA, Mars Climate Orbiter Mishap Investigation Board Phase I Report, November 10, 1999

A Referee Checks Every Play Against the Rulebook

ONE PLAY

Review one play at a time, in order.

RULEBOOK

Check that play against the rules.

CALL IT

Wave it through — or name exactly what broke.

CHECK FOR UNDERSTANDING

In the referee analogy, what equals one batch-file line?

A The rulebook

B The referee

C One play

ANSWER KEY

Referee Analogy Recall

Correct answer: One play

Each play is checked individually against the rules — just like each batch-file line is validated on its own.

Validate One Line in Four Moves

1

SPLIT

Break the line into fields

2

SHAPE

Check the field count

3

VALUE

Check field contents

4

REPORT

Say exactly what failed

`enumerate()` Adds the Position You Need

BEFORE

Output shows the bad line's text.

`enumerate()` Adds the Position You Need

BEFORE

Output shows the bad line's text.

PROBLEM

430 lines are too many to scan.

`enumerate()` Adds the Position You Need

BEFORE

Output shows the bad line's text.

PROBLEM

430 lines are too many to scan.

FIX

`enumerate()` adds each line's position.

Open the Code-Along

Open `Module1_Lesson9_writing-your-first-validation-function-from-scratch_STUDENT.py`
Run it after every section: `python3` + the file name

Sections, in order:

1. Step 1: Just Look at the Data
   Every step-by-step build starts with the smallest possible piece: just looking at the data.
2. Step 2: Check Each Line's Shape
3. Step 3: Wrap It in a Real Function
   Now make it reusable: the same loop and conditional, moved inside a real function.
4. Step 4: Add `enumerate()`
   Swap the plain loop for enumerate() so the function can report human-readable line numbers.
5. Putting It Together: Both Checks at Once
   A correct field count doesn't guarantee the value is actually a number: add the second check.

Step 1: Just Look at the Data

```python
lines = [
    "ADD,Chess,4",
    "ADD,Checkers,two",
    "ADD,Uno",
    "",
    "ADD,Clue,6",
]

for line in lines:
    print(line)
```

Step 2: Check Each Line's Shape

```python
for line in lines:
    fields = line.split(",")
    if len(fields) == 3:
        print("Looks valid:", line)
    else:
        print("Wrong number of fields:", line)
```

Step 3: Wrap It in a Real Function

python

```python
def validate_batch(lines):
    for line in lines:
        fields = line.split(",")
        if len(fields) == 3:
            print("Looks valid:", line)
        else:
            print("Wrong number of fields:", line)

validate_batch(lines)
```

Step 4: Add `enumerate()`

```python
def validate_batch(lines):
    for line_number, line in enumerate(lines,
start=1):
        fields = line.split(",")
        if len(fields) == 3:
            print(f"Line {line_number}: looks valid")
        else:
            print(f"Line {line_number}: wrong number
of fields")

validate_batch(lines)
```

Putting It Together: Both Checks at Once

```python
def validate_batch(lines):
    for line_number, line in enumerate(lines, 1):
        fields = line.split(",")
        if len(fields) != 3:
            print(f"Line {line_number}: bad format")
        elif not fields[2].isdigit():
            print(f"Line {line_number}: bad players")
        else:
            action, title, players = fields
            print(f"Line {line_number}: OK, {title}")
validate_batch(lines)
```

CODE-ALONG DONE

Back to the Slides: Review

You just typed it and ran it. Now check that it stuck, without the code in front of you.

QUICK CHALLENGE

What does Step 2 print for `"ADD,Uno"`?

A

`"Wrong number of fields: ADD,Uno"`

B

`"Looks valid: ADD,Uno"`

C

Nothing — it errors

ANSWER KEY

Step 2 Trace

Correct: `"Wrong number of fields: ADD,Uno"`

`"ADD,Uno".split(",")` gives only 2 items, so it fails `len(fields) == 3`.

QUICK CHALLENGE

`enumerate(lines)` has no `start=1`. What's wrong?

A Nothing

B Line numbers start at 0

C They count backwards

ANSWER KEY

`enumerate()` Off-by-One Bug

Correct: Line numbers start at 0

`enumerate()` defaults to 0. For line numbers that match a text editor, set `start=1` explicitly.

QUICK CHALLENGE

Which check in the Putting It Together code flags `"ADD,Checkers,two"`?

A The `len(fields)` check

B The unpacking line

C The `isdigit()` check

ANSWER KEY

Which Check Fired?

Correct: The `isdigit()` check

It has 3 fields, so the `len(fields)` check passes. But `"two".isdigit()` is `False`, so the `elif` flags it.

Watch It Run: Which Check Fires

Step 1 of 6

We call the function with one line: ADD,Checkers,two

```
def validate_batch(lines):
    for line_number, line in enumerate(lines, 1):
        fields = line.split(",")
        if len(fields) != 3:
            print(f"Line {line_number}: bad format")
        elif not fields[2].isdigit():
            print(f"Line {line_number}: bad players")
validate_batch(["ADD,Checkers,two"])
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

TIP

“How Does Your Validator Handle a Short Line?”

THE LINE

`"ADD,Uno"`

THE CODE

`validate_batch`, Putting It Together

Code Walkthrough:
Inputs, Control Flow, State, Output, Edge Case, Improvement

MODEL ANSWER

Model Answer: Code Walkthrough

INPUTS & CONTROL FLOW

A list of lines; `enumerate` numbers each one, then splits it.

STATE & OUTPUT

Only 2 fields, so it prints the bad-format message for line 3.

EDGE CASE

Checking `fields[2]` first would raise `IndexError` on this line.

IMPROVEMENT

Return the bad line numbers instead of only printing them.

ADD TO THE BOARD

Build the Function in This Order

1

Define the function

2

Loop with `enumerate()`

3

Split fields

4

Check fields

Key Terms — Recap

Batch file

One record per line

Validation

Check data against rules

`enumerate()`

Add each line's position

So Far, We've Covered…

TODAY

A real validation function, built from scratch.

PROJECT

This is the project's `validate_line`.

NEXT

Handle bad input without crashing.